



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Душан Ђорђевић

Континуирана интеграција и испорука сервиса без серверских инстанци на AWS платформи

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Душан Ђорђевић	Број индекса:	SV 1/2021
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Мирослав Зарић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Континуирана интеграција и испорука сервиса без серверских инстанци на AWS платформи
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Душан Ђорђевић
Ментор, МН:	Др Мирослав Зарић, редовни професор
Наслов рада, НР:	Континуирана интеграција и испорука сервиса без серверских инстанци на AWS платформи
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	6/40/19/0/17/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst .
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Мирослав Зарић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Dušan Đorđević
Mentor, MN :	Miroslav Zarić, Phd., full professor
Title, TI :	Continuous Integration and Continuous Delivery of serverless services on the AWS platform
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/40/19/0/17/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Miroslav Zarić, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Структура рада	2
2	Теоријске основе	3
2.1	Рачунарство у облаку	3
2.2	Архитектура без сервера	4
2.3	Инфраструктура као код	6
2.4	Континуирана интеграција и континуирана испорука	7
3	Функционални и нефункционални захтеви	11
3.1	Функционални захтеви	11
3.2	Нефункционални захтеви	12
4	Архитектура	13
4.1	Архитектура <i>streaming</i> апликације	13
4.2	<i>CI/CD pipeline</i> архитектура	17
5	Имплементација	21
5.1	Структура пројекта и <i>CDK</i> апликација	21
5.2	Имплементација <i>CI/CD Pipeline Stack</i> -а	22
5.3	Имплементација система обавештавања	25
6	Закључак	27
6.1	Предности имплементираног решења	27
6.2	Препознати недостаци	27
6.3	Правци даљег развоја	27
	Списак слика	29
	Списак листинга	31
	Списак коришћених скраћеница	33
	Списак коришћених појмова	35
	Биографија	37
	Литература	39

Данашњи софтверски системи постају све комплекснији [1], а захтеви корисника и тржишта условљавају брзу испоруку нових функционалности и побољшања. Због честих ажурирања и развоја нових верзија апликација, неопходно је обезбедити поуздан, аутоматизован и поновљив процес постављања (енгл. *deployment*) на различита окружења [2].

Ручно постављање софтвера је споро, подложно грешкама и тешко одрживо у великим тимовима и производним системима [3]. Управо зато јавља се потреба за механизмима континуиране интеграције и континуиране испоруке (енгл. *Continuous Integration/Continuous Delivery, CI/CD*), који омогућавају аутоматизацију процеса изградње, тестирања и постављања софтверских решења [2].

Додатни ниво комплексности настаје у системима базираним на архитектури без сервера (енгл. *serverless*) [4], где велики број компоненти мора бити правилно повезан, конфигурисан и ажуриран. У таквим системима апликација се састоји од више сервиса у облаку чије ручно одржавање представља значајан изазов.

У AWS окружењу, *serverless* архитектура омогућава скалабилност, високу доступност и смањене оперативне трошкове [4], али истовремено захтева пажљиво осмишљен процес аутоматизованог постављања и конфигурисања инфраструктуре. Апликација у оваквом систему није дефинисана искључиво изворним кодом, већ и инфраструктуром која је описана као код (енгл. *Infrastructure as Code, IaC*) [5]. Функционалност система директно зависи од конфигурације ресурса у облаку (енгл. *cloud*), као што су функције, дозволе приступа, *API* руте и међусобне зависности сервиса, због чега су измене у инфраструктури подједнако значајне као и измене у апликационом коду.

У таквом контексту, примена *CI/CD* механизма мора обухватити аутоматизовано управљање и кодом и инфраструктуром [6]. Недоследно или ручно ажурирање *cloud* ресурса може довести до неконзистентности између окружења, безбедносних пропуста или прекида у раду система. Добро дефинисан *CI/CD* процес (енгл. *pipeline*) омогућава контролисане, верзионисане и поновљиве измене целокупног система, као и брз опоравак у случају грешке [3].

Значај *CI/CD* приступа посебно долази до изражаја код великих дистрибуираних система [7], као што су сервиси за репродукцију садржаја у реалном времену (енгл. *streaming*) слични платформама *Netflix*, *Amazon Prime Video* или *Spotify*. Овакве апликације карактерише велики број корисника, високи захтеви за доступношћу и континуирано увођење нових функционалности [8]. Због ослањања на микросервисну и *serverless* архитектуру [9], [10], компоненте се развијају и ажурирају паралелно, што захтева поуздан механизам за учестале, мале и контролисане измене без прекида рада система [11].

Имајући у виду наведене захтеве, предмет овог рада је имплементација *CI/CD* процеса за већ постојећи *streaming* сервис заснован на *AWS serverless* архитектури. Циљ је да се прикаже како унапређење и формализација *CI/CD pipeline*-а доприносе скалабилности, поузданости и дугорочној одрживости сложених *cloud* апликација.

1.1 Структура рада

Овај рад је организован у више логички повезаних поглавља која постепено уводе читаоца у проблематику, од теоријских основа до практичне имплементације *CI/CD* процеса за *AWS serverless* апликацију.

Друго поглавље обухвата теоријске основе неопходне за разумевање рада. У овом поглављу објашњени су појмови као што су рачунарство у облаку, *CI/CD*, *serverless* архитектура, као и концепт инфраструктуре као кода. Такође је дат преглед релевантних *AWS* сервиса који чине основу имплементираног решења.

У трећем поглављу дефинисани су функционални и нефункционални захтеви система. Описани су захтеви који се односе на поузданост, безбедност, скалабилност, доступност и одрживост система, као и захтеви који произилазе из природе *streaming* сервиса и *CI/CD* процеса.

Четврто поглавље посвећено је архитектури система и опису постојећег *streaming* сервиса. У овом поглављу приказана је укупна архитектура *AWS serverless* система, структура *cloud* ресурса и начин њихове међусобне интеракције. Такође су описани *AWS* сервиси и *stack*-ови који су коришћени, као и *CI/CD* алати и сервиси примењени за изградњу *pipeline*-а.

Пето поглавље представља централни део рада и приказује конкретну имплементацију *CI/CD pipeline*-а. Описани су кораци *pipeline*-а, аутоматизација изградње, тестирања и постављања апликационог кода и инфраструктуре, као и механизми за контролу верзија и опоравак у случају грешке.

На крају рада дат је закључак у којем су сумирани постигнути резултати, анализирани ефекти примене *CI/CD* приступа и предложени правци за даље унапређење система.

2.1 Рачунарство у облаку

Рачунарство у облаку (engl. *cloud computing*) представља модел пружања рачунарских ресурса преко интернета. Пример таквих ресурса могу бити процесорска снага, складиштење података и апликације. Корисници могу приступити овим ресурсима по потреби, без потребе за поседовањем или одржавањем физичке инфраструктуре. Овакав модел омогућава лако скалирање капацитета у складу са захтевима, што повећава флексибилност и оптимизује трошкове оперативног рада. Окружење у облаку је дистрибуирано и редундантно, што значајно повећава поузданост и доступност система чак и у случају хардверских или софтверских отказа.

Поред тога, рачунарство у облаку омогућава брзу имплементацију и тестирање нових услуга, јер се инфраструктура може конфигурисати и модификовати без дуготрајне набавке или инсталације опреме. Централизована контрола и мониторинг ресурса омогућавају бољу оптимизацију коришћења и лакше управљање сложеним системима. Захваљујући овој флексибилности и поузданости, *cloud* модели представљају основ за модерне апликације које захтевају високу доступност, брзе испоруке промена и сигурност података.

2.1.1 Модели услуга у рачунарству у облаку

У рачунарству у облаку постоје три основна модела услуга који обједињују различите нивое управљања и одговорности корисника и провајдера: инфраструктура као сервис, платформа као сервис и софтвер као сервис. Ови модели омогућавају организацијама да делимично или у потпуности препусте управљање инфраструктуром и софтверским слојевима трећој страни, чиме се смањује потреба за сопственим центром података и одржавањем сопствене рачунарске опреме.

2.1.1.1 Инфраструктура као сервис

Инфраструктура као сервис (енгл. *Infrastructure-as-a-service, IaaS*) је модел у коме провајдер пружа основну инфраструктуру као услугу — укључујући рачунарске ресурсе, складиштење, мрежу и виртуелизацију — док корисник управља оперативним системом, апликацијама и подацима. Ово омогућава велико прилагођавање и скалабилност без сопственог хардвера и физичког одржавања.

Код *IaaS* модела, корисник задржава висок ниво контроле над конфигурацијом система, што омогућава прилагођавање различитим типовима апликација и специфичним захтевима. Овај модел се често користи у ситуацијама када је потребна потпуна контрола над окружењем, али без трошкова и сложености управљања физичком инфраструктуром [12].

2.1.1.2 Платформа као сервис

Платформа као сервис (енгл. *Platform-as-a-service, PaaS*) нуди не само инфраструктуру, већ и комплетну платформу за развој и извршавање апликација, укључујући потребне алате и окружења. Корисник се фокусира на развој и управљање апликацијама, док провајдер брине о инфраструктури и платформском софтверу.

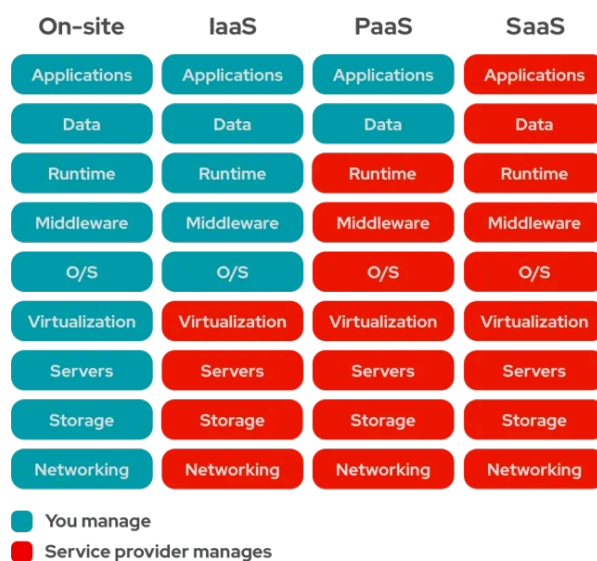
PaaS модел значајно поједностављује процес развоја, јер уклања потребу за ручним подешавањем и одржавањем оперативних система и основних сервиса. Овакво окружење омогућава бржи развој, тестирање и испоруку апликација, што га чини погодним за тимове који желе да се усредсреде искључиво на функционалност и квалитет софтвера [12].

2.1.1.3 Софтвер као сервис

Софтвер као сервис (енгл. *Software-as-a-service*, *SaaS*) је модел у коме провајдер испоручује готову апликацију преко интернета, а корисници приступају софтверу без потребе за инсталацијом или одржавањем. Провајдер управља свим слојевима — од инфраструктуре до самог софтвера — што корисницима омогућава једноставан приступ и употребу апликација.

Овај модел омогућава корисницима да користе сложене софтверске системе без техничког знања о њиховој имплементацији и одржавању. *SaaS* решења су посебно погодна за широк круг корисника, јер омогућавају брзо увођење у рад, аутоматска ажурирања и високу доступност услуга [12].

Разлике између *IaaS*, *PaaS* и *SaaS* модела могу се јасно уочити кроз поделу одговорности између корисника и провајдера услуга, што је приказано на слици 1.



Слика 1: Поређење *IaaS*, *PaaS* и *SaaS* модела према нивоу управљања ресурсима. [12].

2.2 Архитектура без сервера

Архитектура без сервера (енгл. *serverless architecture*) представља модел извршавања апликација у коме *cloud* провајдер у потпуности управља како алокацијом тако и алоцираним рачунарским ресурсима. Иако се апликације и даље извршавају на серверима, корисник није одговоран за њихово конфигурисање, скалирање или одржавање, већ се фокусира искључиво на имплементацију апликационе логике [13].

Основна идеја *serverless* приступа заснива се на извршавању апликације искључиво у тренутку када је то потребно, при чему се ресурси динамички додељују и ослобађају након завршетка извршавања. Наплата се врши на основу стварно утрошених ресурса и времена извршавања, што омогућава оптимизацију трошкова у односу на традиционалне моделе где се ресурси резервишу унапред.

Serverless архитектура се најчешће реализује кроз модел функција које се извршавају као реакција на одређене догађаје, као што су *HTTP* захтеви, промене у систему или поруке из других сервиса. Апликација се логички разлаже на мање, независне и, најчешће, компоненте које не памте стање, што омогућава високу скалабилност и паралелно извршавање великог броја захтева.

У овом моделу, већина оперативних одговорности прелази на *cloud* провајдера, укључујући управљање центрима података, серверима, оперативним системима и извршним окружењима. Оваква подела одговорности значајно смањује сложеност развоја и одржавања система, али истовремено уводи нове изазове као што су сложеније управљање конфигурацијама, ограничена контрола над извршним окружењем и већа зависност од провајдера услуга.

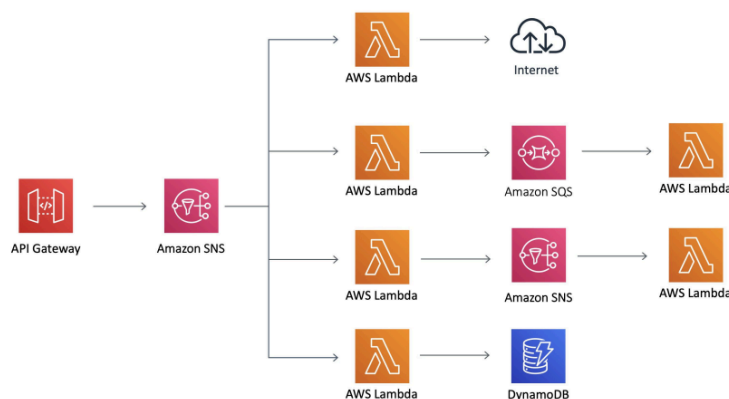
Због своје флексибилности, аутоматског скалирања и ефикасног коришћења ресурса, *serverless* архитектура је постала важан део савремених *cloud* система, посебно у апликацијама које захтевају високу доступност, динамичко оптерећење и брзу испоруку функционалности.

2.2.1 AWS архитектура без сервера

AWS архитектура без сервера представља практичну примену *serverless* концепта у оквиру AWS *cloud* платформе, где провајдер преузима потпуну одговорност за управљање инфраструктуром, а корисник се фокусира на апликациону логику и конфигурацију система. Основна идеја овакве архитектуре заснива се на догађајно оријентисаном моделу, у коме се поједине компоненте система активирају као одговор на одређене догађаје, без потребе за стално активним серверима.

Централни елемент AWS *serverless* архитектуре представља сервис AWS *Lambda*, који омогућава извршавање апликационе логице у облику функција. Ове функције се аутоматски покрећу као одговор на различите типове догађаја, при чему AWS управља скалирањем, доступношћу и извршним окружењем, а наплата се врши на основу времена извршавања и утрошених ресурса.

За асинхрону комуникацију и размену порука (о догађајима у систему) између компоненти система, AWS обезбеђује сервисе као што су *Amazon Simple Notification Service (SNS)* и *Amazon Simple Queue Service (SQS)*. *SNS* представља сервис за објављивање порука у моделу „један-према-више“, где се порука (информација о догађају) шаље једном и аутоматски дистрибуира свим регистрованим претплатницима. Овакав приступ је погодан за обавештавање више компоненти система о истом догађају без међусобне зависности. Са друге стране, *SQS* је сервис за поуздано редно складиштење порука који обезбеђује да свака порука буде безбедно сачувана и обрађена од стране тачно једног потрошача. *SQS* омогућава раздвајање произвођача и потрошача порука, као и контролу над поновним покушајима обраде у случају грешке, чиме се обезбеђује поуздано и на грешке отпорно асинхронно извршавање. Комбинацијом *SNS* и *SQS* сервиса могуће је реализовати архитектуре које истовремено омогућавају дистрибуцију догађаја и гарантовано достављање порука.



Слика 2: Пример AWS *serverless* архитектуре [14].

Оваква архитектура подстиче развој апликација које су састављене од мањих, независних и најчешће *stateless* компоненти (без памћења стања), што омогућава високу скалабилност и паралелно извршавање великог броја захтева. Истовремено, већина оперативних одговорности, као што су управљање серверима, оперативним системима и мрежном инфраструктуром, у потпуности је апстрахована од корисника и преузета од стране AWS платформе.

Иако AWS *serverless* архитектура значајно поједностављује развој и одржавање система, она уводи и одређене изазове, као што су сложеније управљање конфигурацијама, праћење извршавања дистрибуираних

компоненти и зависност од конкретних AWS сервиса. Због тога је неопходно пажљиво осмишљавање архитектуре и примена аутоматизованих механизма за постављање, надзор и контролу система.

2.3 Инфраструктура као код

Инфраструктура као код (енгл. *Infrastructure as Code – IaC*) представља приступ управљању инфраструктуром у коме се конфигурација рачунарских ресурса дефинише помоћу формалних, машински читљивих фајлова [5]. Уместо ручног креирања и конфигурисања ресурса, целокупна инфраструктура се описује декларативно или процедурално, што омогућава њено аутоматизовано креирање, измену и уклањање.

Основна идеја IaC приступа јесте примена принципа софтверског инжењерства на управљање инфраструктуром [6]. Инфраструктурне дефиниције се чувају у систему за контролу верзија, што омогућава праћење историје промена, контролу приступа и лако враћање на претходне стабилне конфигурације. Оваква пракса значајно смањује ризик од неконзистентности између окружења и грешака које настају услед ручних интервенција.

Примена инфраструктуре као кода омогућава висок степен аутоматизације и репродуктивности [5]. Једном дефинисана инфраструктура може се више пута креирати на идентичан начин у различитим окружењима, као што су развојно, тестно и продукционо. Ово је посебно значајно у *cloud* окружењима, где се ресурси динамички креирају и уклањају у складу са потребама система.

Поред тога, IaC приступ подстиче бољу сарадњу између развојних и оперативних тимова, јер инфраструктурне промене постају транспарентне и део истог процеса као и развој апликационог кода [6]. Аутоматизација инфраструктуре доприноси бржем постављању система, лакшем тестирању промена и поузданијем управљању сложеним дистрибуираним архитектурама.

У контексту савремених *cloud* и *serverless* система, инфраструктура као код представља кључни механизам за изградњу стабилних, скалабилних и одрживих решења [5]. Њена примена омогућава интеграцију инфраструктурних промена у *CI/CD* процесе, чиме се обезбеђује конзистентност и поузданост целокупног животног циклуса апликације.

2.3.1 AWS инфраструктура као код

У AWS окружењу, концепт инфраструктуре као кода реализује се кроз више алата и сервиса који омогућавају декларативно и програмско дефинисање *cloud* ресурса. AWS пружа изворну подршку за IaC приступ кроз сервис *AWS CloudFormation*, који омогућава описивање инфраструктуре у виду шаблона. Ови шаблони дефинишу ресурсе, њихове међусобне зависности и конфигурације, чиме се омогућава аутоматизовано и поновљиво постављање инфраструктуре у различитим окружењима.

CloudFormation представља основу *AWS IaC* екосистема, али рад са великим и сложеним шаблонима може постати тешко одржив, посебно у системима који се састоје од великог броја ресурса. Због тога AWS нуди *AWS Cloud Development Kit (CDK)*, који представља виши ниво апстракције над *CloudFormation* механизмом. *AWS CDK* омогућава дефинисање инфраструктуре коришћењем познатих програмских језика, као што су *TypeScript*, *Java*, *Python* или *C#*, уместо директног писања шаблона.

Коришћењем *AWS CDK*-а, инфраструктура постаје део апликационог кода и може користити програмске концепте као што су апстракција, поновна употреба компоненти и модуларност. *CDK* код се током процеса изградње трансформише у *CloudFormation* шаблоне, који се затим користе за стварно постављање ресурса у *AWS* окружењу. На овај начин задржавају се предности *CloudFormation*-а, као што су управљање стањем и контролисане измене, уз значајно повећање читљивости и одрживости кода.

Примена *AWS IaC* алата, посебно у комбинацији са *CI/CD* процесима, омогућава потпуну аутоматизацију управљања инфраструктуром. Све измене инфраструктуре пролазе кроз исти процес верификације, тестирања и одобравања као и апликациони код, чиме се смањује ризик од грешака и неконзистентности. Овакав

приступ је од посебне важности у *serverless* и микросервисним архитектурама, где се инфраструктура често и динамички мења.

Захваљујући приступу инфраструктури у виду кода и алатима као што су *AWS CloudFormation* и *AWS CDK*, могуће је изградити стабилна, скалабилна и репродуктивна *cloud* окружења, која се лако прилагођавају променама захтева и даљем развоју система.

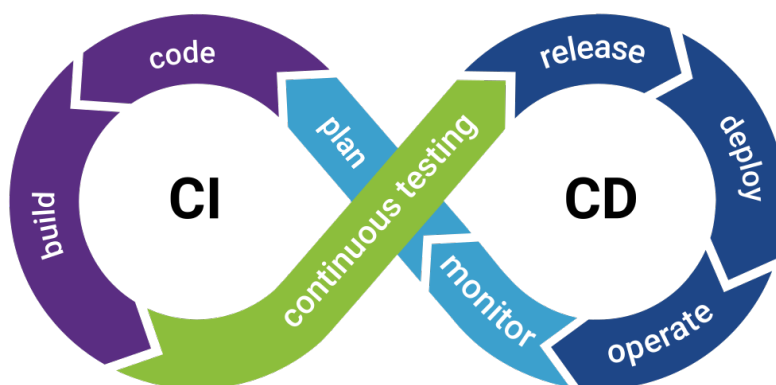
2.4 Континуирана интеграција и континуирана испорука

Континуирана интеграција (енгл. *Continuous Integration, CI*) и континуирана испорука (енгл. *Continuous Delivery, CD*) представљају скуп пракси које имају за циљ аутоматизацију и унапређење процеса развоја и испоруке софтвера. Континуирана интеграција подразумева често интегрисање измена у заједничку кодну базу, при чему се свака измена аутоматски гради и тестира како би се грешке откриле у раној фази развоја [2].

Основни циљ континуиране интеграције је смањење ризика који настаје услед дуготрајне изоловане измене кода, као и елиминација сложених и непредвидивих интеграционих фаза [15]. Аутоматизацијом грађења и тестирања обезбеђује се брза повратна информација програмерима, што доприноси стабилнијем и квалитетнијем развоју система [3].

Континуирана испорука представља проширење *CI* праксе и фокусира се на аутоматизацију процеса припреме софтвера за постављање у различита окружења [15]. У оквиру *CD* приступа, свака успешно тестирана верзија апликације је у стању да буде испоручена у продукционо окружење у било ком тренутку, уз минималну ручну интервенцију.

У неким системима примењује се и континуирано постављање (енгл. *Continuous Deployment*), где се свака промена која прође све фазе *pipeline*-а аутоматски поставља у продукцију [2]. Овај приступ омогућава веома честе испоруке, али захтева висок ниво аутоматизације, тест покривености и надзора.



Слика 3: *CI/CD* циклус [16].

Као што је приказано на слици 3, *CI/CD* процес представља континуиран циклус који интегрише развој и оперативне активности. Лева страна циклуса (*CI*) обухвата планирање развоја, писање кода, аутоматско изградњу инсталационих фајлова, док десна страна (*CD*) покрива фазе испоруке, постављања (инсталације у извршно окружење) и оперисања система у продукцији. Континуирано тестирање (енгл. *continuous testing*) и надзор (енгл. *monitoring*) повезују оба процеса, обезбеђујући повратну информацију која омогућава брзе исправке и континуирано унапређење квалитета софтвера [15].

Правилан дизајн *CI/CD pipeline*-а обухвата јасно дефинисане фазе изградње, тестирања и постављања. Примена проверених образаца за скалабилност, поузданост и безбедност омогућава изградњу стабилних система за испоруку софтвера у сложеним дистрибуираним окружењима [3], [15].

2.4.1 Фазе CI/CD pipeline-a

Иако постоји *ISO/IEC/IEEE 32675:2022(E)* стандард који дефинише DevOps праксе, не постоји јединствена стандардна дефиниција за фазе *CI/CD pipeline-a* [17]. Свака организација прилагођава своје *pipeline*-ове специфичним потребама пројекта, технолошком стеку и пословним захтевима [15]. У наставку су описане уобичајене фазе које чине основу већине *CI/CD* имплементација.

Фазе континуиране интеграције (CI)

Континуирана интеграција обухвата фазе од промене кода до креирања тестираног артефакта спремног за испоруку [2].

1. Верзионисање изворног кода

Процес почиње постављањем измена у систем за контролу верзија попут *Git*-а [15]. Сваки *commit* или *pull request* иницира аутоматско извршавање *pipeline-a*. Употреба стратегија гранања, као што су *GitFlow* или *trunk-based development*, омогућава ефикасан паралелан развој и контролисану интеграцију измена [2]. Ова фаза често укључује и иницијалне провере квалитета кода, попут *linting*-а и провере синтаксе.

2. Грађење (Build)

Фаза грађења трансформише изворни код у извршне артефакте [3]. У зависности од типа апликације, ова фаза може укључивати компилацију кода, резолуцију зависности, паковање у *JAR/WAR* фајлове, креирање *Docker* слика или припрему *serverless* функција. За *AWS serverless* апликације, алати попут *AWS SAM*, *Serverless Framework* или *Terraform* аутоматизују процес паковања функција и инфраструктурних компоненти [18]. Фаза грађења такође генерише логове и обавештава програмере о успеху или неуспеху процеса.

3. Аутоматизовано тестирање

Након успешног грађења, извршавају се аутоматизовани тестови како би се рано откриле грешке [3]. Тестирање обично почиње јединичним тестовима (*unit tests*) који валидују појединачне компоненте у изолацији, након чега следе интеграциони тестови који проверавају интеракцију између различитих модула [15]. У *serverless* окружењима, тестирање укључује симулацију догађаја који покрећу функције и проверу интеграције са *AWS* сервисима попут *DynamoDB*, *S3* или *API Gateway*. Резултати тестова одређују да ли процес може наставити ка наредној фази или се враћа програмерима на корекцију.

4. Анализа квалитета кода

Алати за статичку анализу кода процењују квалитет, одржавање и безбедност кодне базе [15]. Ова фаза детектује потенцијалне проблеме као што су дупликација кода, неусаглашеност са стандардима, безбедносне рањивости и *code smells*. Раном откривањем ових проблема, програмери могу унапредити квалитет апликације пре него што код стигне до производње.

Фазе континуиране испоруке и постављања (CD)

Континуирана испорука преузима верзију која је прошла *CI* фазу и припрема је за испоруку у различита окружења [2].

1. Постављање на развојно/тест окружење

Артефакти који су успешно прошли све *CI* фазе аутоматски се постављају у развојно (енгл. *development*) или тестно (енгл. *staging*) окружење [15]. Ова фаза омогућава извршавање додатних интеграционих тестова, проверу перформанси и валидацију у окружењу које симулира продукцију.

2. Валидација инфраструктуре

Пре продукционог постављања, извршава се валидација инфраструктурних измена дефинисаних као код [18]. Алати попут *Terraform plan* или *AWS CloudFormation change sets* приказују промене које ће бити примењене на *cloud* ресурсе. Ова фаза омогућава преглед и одобрење инфраструктурних измена пре њиховог извршења, чиме се смањује ризик од нежељених последица по производни систем.

3. Одобрење за продукцију

У континуираној испоруци (*Continuous Delivery*), постоји фаза одобрења пре коначног постављања у продукцију [2]. Ово одобрење може бити ручно, где менаџери или тим лидери прегледају промене и доносе одлуку о испоруци, или аутоматизовано на основу унапред дефинисаних критеријума квалитета и безбедности. Код континуираног постављања (*Continuous Deployment*), ова фаза се прескаче и све промене које прођу претходне фазе аутоматски се постављају у продукцију [15].

4. Постављање у продукционо окружење

У овој фази апликација се коначно поставља у продукционо окружење [3]. Стратегије постављања укључују:

- *Blue-green deployment*: паралелно постављање нове верзије поред постојеће, са тренутном променом саобраћаја
- *Canary releases*: постепено усмеравање малог процента корисника на нову верзију пре потпуног постављања
- *Rolling updates*: поступна замена инстанци старе верзије новом

Ове стратегије минимизују ризик и омогућавају брз опоравак (енгл. *rollback*) у случају проблема [3].

5. Мониторинг и повратна информација

Након успешног постављања, систем се континуирано прати помоћу метрика, логова и аларма. AWS сервиси попут *CloudWatch*, *X-Ray* и *CloudTrail* пружају увид у перформансе, грешке, коришћење ресурса и безбедносне догађаје. Повратна информација из мониторинга омогућава:

- брзу детекцију и реакцију на проблеме у продукцији,
- идентификацију узрока застоја или лоших перформанси,
- оптимизацију трошкова *cloud* ресурса,
- унапређење будућих верзија *pipeline*-а на основу реалних података.

2.4.2 AWS CI/CD сервиси

AWS нуди комплетан скуп алата за имплементацију *CI/CD* процеса у облаку, који омогућавају аутоматизацију целог циклуса развоја, тестирања и испоруке апликација. Ови сервиси су дизајнирани да раде заједно и интегришу се са другим AWS сервисима, пружајући флексибилно и скалабилно решење за континуирану испоруку софтвера. AWS нуди више сервиса за подршку *CI/CD* процеса, међу којима су *CodePipeline*, *CodeBuild* и *CodeDeploy*, при чему се конкретан избор и начин употребе сервиса прилагођава архитектури и потребама система.

• AWS CodePipeline

AWS CodePipeline представља оркестрациони сервис који омогућава дефинисање и управљање фазама *CI/CD pipeline*-а. Он аутоматски покреће процес интеграције и испоруке након детекције измена у систему за контролу верзија и обезбеђује редоследно извршавање свих дефинисаних корака. *CodePipeline* омогућава визуелни преглед статуса сваке фазе, као и лако проширење *pipeline*-а додавањем нових корака или интеграцијом са другим AWS и спољним сервисима.

• AWS CodeBuild

AWS CodeBuild је потпуно управљан сервис за аутоматизовану изградњу и тестирање апликација. Он омогућава компилацију изворног кода, извршавање тестова и креирање артефаката без потребе за одржавањем сопствених *build* сервера. *CodeBuild* динамички заузима ресурсе у зависности од захтева процеса изградње, што доприноси скалабилности и оптимизацији трошкова, а конфигурација процеса дефинише се декларативно кроз конфигурационе датотеке.

Глава 3

Функционални и нефункционални захтеви

Функционални и нефункционални захтеви представљају основу за пројектовање и евалуацију софтверског система. Функционални захтеви дефинишу конкретне функције и понашање система, док нефункционални захтеви описују квалитативне особине као што су поузданост, безбедност, перформансе и одрживост.

3.1 Функционални захтеви

Функционални захтеви дефинишу основне могућности које *CI/CD* систем мора да обезбеди како би подржао развој, тестирање и постављање *streaming* сервиса заснованог на *AWS serverless* архитектури. Ови захтеви описују конкретне функције и понашања система, са циљем аутоматизације процеса, смањења ручних интервенција и обезбеђивања конзистентности између окружења.

3.1.1 Аутоматско покретање *CI/CD* процеса

Систем мора да омогући аутоматско покретање *CI/CD pipeline*-а при свакој измени апликационог кода или инфраструктурних конфигурација у систему за контролу верзија. Ово подразумева реаговање на догађаје као што су *push*, *merge* или креирање нових верзија. Аутоматско покретање процеса смањује ризик од пропуштања измена и обезбеђује да се свака промена доследно обради. На тај начин се обезбеђује брза повратна информација о исправности измена.

3.1.2 Аутоматизована изградња апликације

Систем мора да подржи аутоматизовану изградњу апликационог кода и припрему *serverless* пакета за постављање. Процес изградње обухвата инсталацију зависности, компилацију и паковање функција за *AWS* окружење. Аутоматизација овог корака обезбеђује поновљивост и елиминише разлике између локалних и продукционих окружења. Такође, омогућава рану детекцију грешака у фази изградње.

3.1.3 Аутоматизовано тестирање

Систем мора да омогући извршавање аутоматизованих тестова у оквиру *CI/CD* процеса ради ране детекције грешака. Ово укључује јединичне и интеграционе тестове који проверавају исправност функционалности апликације. Тестови се извршавају аутоматски након фазе изградње и представљају предуслов за даље кораке у *pipeline*-у. На овај начин се смањује ризик од увођења нестабилних верзија у наредна окружења.

3.1.4 Аутоматизовано постављање апликације

Систем мора да обезбеди аутоматизовано постављање апликационог кода у развојно окружење без ручне интервенције, након успешног завршетка фазе изградње и тестирања. За тестно и продукционо окружење, процес постављања мора да укључује механизам ручног одобравања. Оваквим приступом омогућена је брза испорука промена током развоја, уз задржавање контроле и стабилности у окружењима која су критична за квалитет и доступност система. Комбинација аутоматизације и ручне валидације доприноси смањењу ризика и унапређењу поузданости целокупног *CI/CD* процеса.

3.1.5 Управљање инфраструктуром као кодом

Систем мора да омогући аутоматизовано постављање и ажурирање *cloud* инфраструктуре применом инфраструктуре као кода. Инфраструктурне компоненте, као што су функције, дозволе, *API* руте и догађаји, дефинишу се кроз конфигурационе фајлове. Оваквим приступом инфраструктура постаје верзионисана, поновљива и проверљива. Измене инфраструктуре се третирају на исти начин као и измене апликационог кода.

3.1.6 Подршка за више окружења

Систем мора да омогући раздвајање и независно управљање развојним, тестним и продукционим окружењем. Свако окружење мора имати сопствену конфигурацију и ресурсе, уз контролисан прелаз између њих. Оваква подршка омогућава безбедно тестирање нових функционалности пре продукционог постављања. Такође, доприноси очувању стабилности продукционог система.

3.1.7 Надзор и проширивост *CI/CD* процеса

Систем мора да омогући увид у статус извршавања *CI/CD pipeline*-а и резултате сваког корака. Ово укључује информације о успешности или неуспешности изградње, тестова и постављања. Поред тога, *CI/CD* процес мора бити пројектован тако да омогући једноставно проширење додавањем нових корака, провера или интеграција са другим алатима. Оваква флексибилност омогућава прилагођавање система будућим захтевима и расту сложености апликације.

3.2 Нефункционални захтеви

Нефункционални захтеви дефинишу квалитативне особине *CI/CD* система и одређују начин на који систем функционише, независно од конкретних функционалности. Они су од кључног значаја за поузданост, безбедност и дугорочну одрживост *CI/CD* процеса у *cloud* и *serverless* окружењу.

3.2.1 Поузданост

CI/CD систем мора да обезбеди стабилно и предвидиво извршавање свих фаза *pipeline*-а, уз минималну вероватноћу отказа. У случају грешке, систем мора омогућити јасну идентификацију узрока и безбедан опоравак.

3.2.2 Безбедност

Приступ *CI/CD* процесу и *AWS* ресурсима мора бити заштићен применом механизма аутентификације и ауторизације заснованих на принципу најмањих привилегија. Сви осетљиви подаци, као што су тајне и приступни кључеви, морају бити безбедно складиштени и коришћени.

3.2.3 Конзистентност окружења

Сва окружења морају бити конфигурисана на идентичан начин применом инфраструктуре као кода. Оваква конзистентност смањује ризик од различитог понашања система између развојног, тестног и продукционог окружења.

3.2.4 Аутоматизација

CI/CD процес мора бити максимално аутоматизован како би се смањила зависност од ручних интервенција и могућност људске грешке. Ручно одобравање је дозвољено искључиво у контролисаним фазама, као што је постављање у продукционо окружење.

3.2.5 Репродуктивност

Свака верзија апликације и инфраструктуре мора бити репродуктивна на основу верзионисаних дефиниција у систему за контролу верзија. Овај захтев омогућава поуздано понављање постављања и брз повратак на претходне стабилне верзије.

3.2.6 Перформансе

Време извршавања *CI/CD pipeline*-а мора бити оптимизовано како би се омогућила брза повратна информација током развоја. Ефикасан *pipeline* директно доприноси повећању продуктивности развојног тима.

У овом поглављу представљена је архитектура система која обухвата две кључне компоненте: архитектуру *streaming* апликације и архитектуру *CI/CD pipeline*-а који аутоматизује њено постављање. Архитектура целокупног система дефинисана је декларативно коришћењем *AWS Cloud Development Kit (CDK)*, који омогућава описивање инфраструктуре као кода и конзистентно постављање свих компоненти.

Разумевање архитектуре апликације неопходно је за дефинисање одговарајућег *CI/CD* процеса, јер комплексност и дистрибуирана природа *serverless* система директно утичу на изазове и захтеве аутоматизације испоруке.

Streaming апликација базирана је на *AWS serverless* архитектури и организована је у више функционалних целина (енгл. *stack*) који се заједно постављају кроз један централизован *CI/CD pipeline*. Овај приступ омогућава конзистентно и координисано постављање свих компоненти система, обезбеђујући да промене у једном *stack*-у не утичу негативно на остале делове апликације. У наставку су детаљно приказане архитектуре главних функционалних компоненти система, након чега следи опис *CI/CD* инфраструктуре која обезбеђује њихово аутоматизовано постављање.

Иако *serverless* и микросервисна архитектура нуди бројне предности, важно је напоменути да овај приступ није увек оптимално решење за све типове апликација. *Amazon Prime Video* тим је 2023. године објавио чланак [19] у којој је миграција њиховог сервиса за надгледање квалитета видеа са дистрибуиране *serverless* архитектуре на монолитну апликацију резултирала смањењем трошкова за 90%. Главни проблеми били су везани за ограничења *AWS Step Functions* сервиса и високе трошкове учесталих позива ка *S3* складишту за привремено чување видео фрејмова између компоненти. Пребацавањем свих компоненти у један процес, тим је елиминисао потребу за *S3* складиштем за привремено чување података, омогућавајући да се трансфер података између компоненти врши директно у меморији. Ова архитектура омогућила је да се скалирање врши вертикално додавањем већих инстанци уместо хоризонталног скалирања великог броја микросервиса, што је за овај тип апликације показало као ефикасније решење. Овај пример илуструје да избор архитектуре мора бити заснован на специфичним захтевима сваке апликације, узимајући у обзир факторе као што су образац саобраћаја, очекивани трошкови и сложеност одржавања.

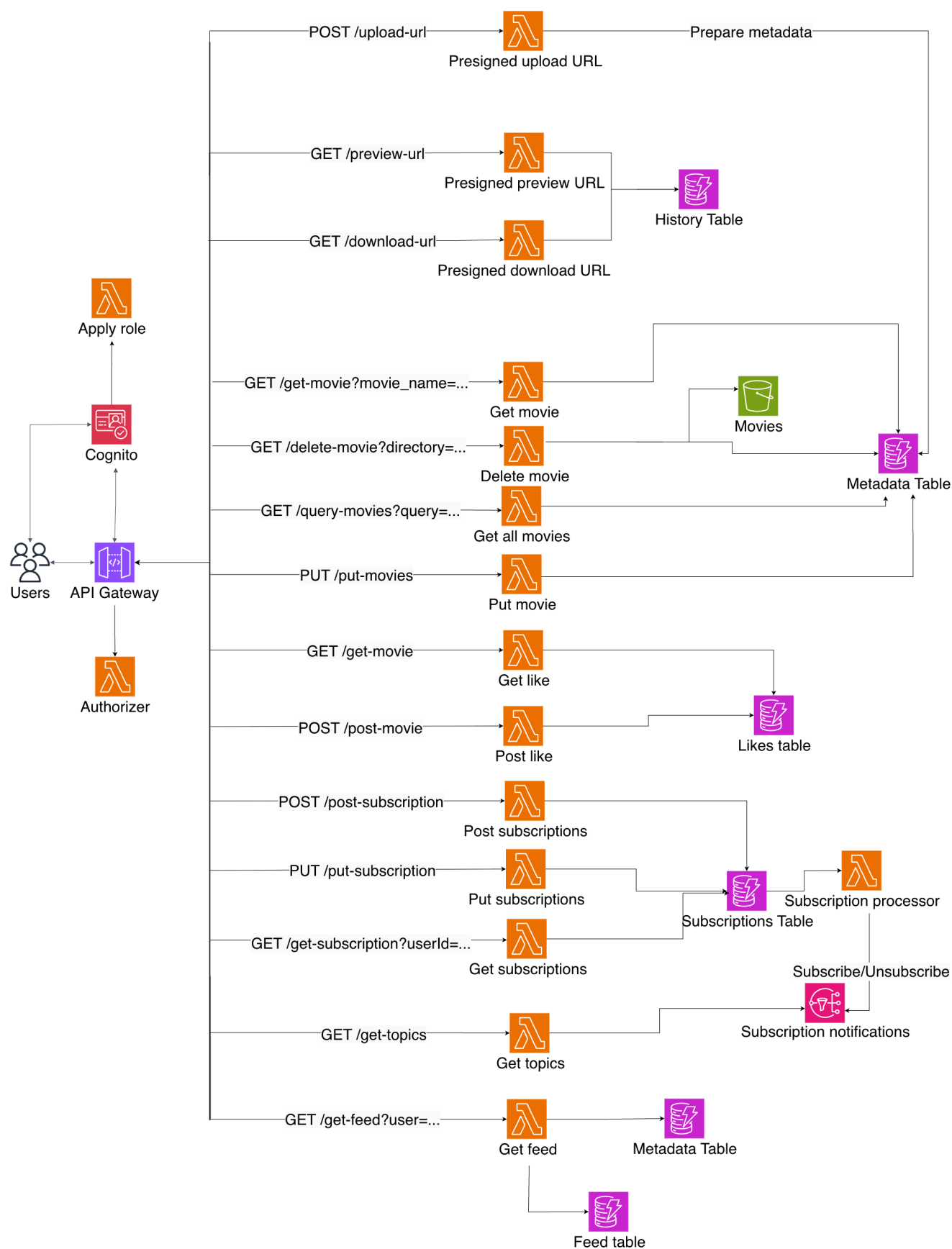
4.1 Архитектура *streaming* апликације

4.1.1 *REST API* и управљање садржајем

Централна компонента система је *REST API* који корисницима омогућава приступ функционалностима апликације. Као што је приказано на слици 4, архитектура је организована око догађајима вођеног модела апликације где *API Gateway* служи као улазна тачка за све захтеве корисника, а *Lambda* функције представљају независне микросервисе који обрађују специфичне типове захтева.

- **Аутентификација и ауторизација**

Систем користи *AWS Cognito* за управљање корисничким идентитетима. Након успешне аутентификације, корисник добија *JWT* токен који се валидира помоћу *Authorizer Lambda* функције при сваком захтеву ка заштићеним ресурсима. *Apply role Lambda* функција додељује одговарајућу улогу након регистрације корисника чиме се обичном кориснику ограничава приступ ресурсима и функцијама.



Слика 4: REST API архітектура.

• Управљање мултимедијалним садржајем

Главни део апликације чине *Lambda* функције за управљање видео садржајем. Систем подржава следеће операције:

Постављање садржаја: *Lambda* функција `upload-url` генерише привремене заштићене адресе (*presigned URL*-ове) који омогућавају корисницима директно постављање филмова у *S3 bucket* без проласка кроз *API Gateway*, чиме се избегавају ограничења величине захтева и оптимизује перформансе. Након што корисник постави филм користећи генерисани *URL*, *S3* догађај (PUT операција) аутоматски покреће *Lambda* функцију која припрема метаподатке и иницира процес транскодирања.

Преглед и преузимање: Функције `preview-url` и `download-url` такође генеришу привремене *URL*-ове за приступ садржају, при чему се метаподаци о приступу складиште у *History Table (DynamoDB)*. Ови подаци касније се користе за генерисање персонализованих препорука у *Feed* систему.

Претраживање и филтрирање: *Lambda* функције `query-movies` и `get-all-movies` омогућавају корисницима претраживање садржаја на основу различитих критеријума (наслов, жанр, режисер) користећи индексе *DynamoDB* табеле *Metadata Table*.

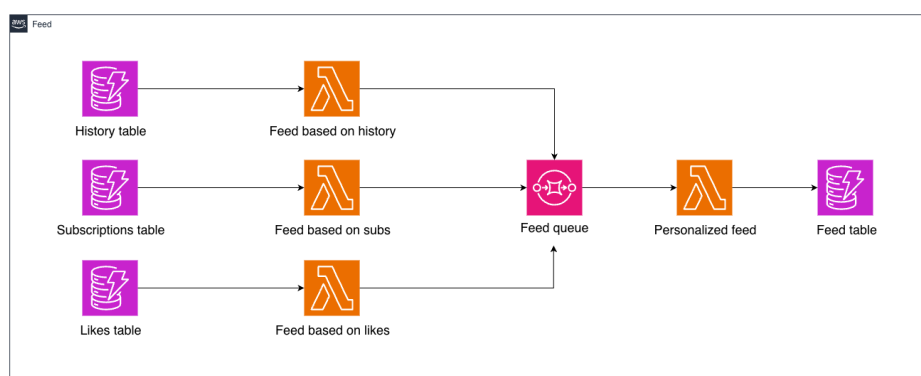
У овој имплементацији, функционалности претраживања реализоване су као одвојене *Lambda* функције, што представља *fine-grained* приступ дизајну са специфичним предностима и недостацима. Алтернативни приступ би био коришћење једне веће *Lambda* функције која на основу типа захтева извршава различите операције у истој функцији. *Fine-grained* приступ омогућава независно скалирање сваке функције, прецизније контролисање дозвола приступа ресурсима и једноставније тестирање и одржавање, али доводи до већег броја *Lambda* функција које треба управљати и потенцијално дуплирање заједничког кода. Са друге стране, монолитније *Lambda* функције смањују број компоненти и олакшавају дељење заједничке логике, али могу постати сложене за одржавање и теже се независно скалирају.

CRUD операције: Функције `put-movies` и `delete-movie` омогућавају ажурирање метаподатака и брисање садржаја. Операција брисања укључује и уклањање физичких фајлова из *S3 bucket*-а и свих повезаних метаподатака из база података.

Сви оригинални филмови и њихове транскодирани верзије складиште се у *Movies S3 bucket*-у организованом у хијерархијску структуру директоријума према резолуцији и формату. Метаподаци о филмовима (наслов, опис, трајање, формат, величина) чувају се у *Metadata Table*, обезбеђујући брзо претраживање и конзистентност података.

4.1.2 Архитектура *Feed* система

Feed систем представља посебну компоненту која генерише јединствен и индивидуалан садржај за сваког корисника на основу његовог понашања и преференци (Слика 5).



Слика 5: Архитектура *Feed* система.

Feed систем користи класичан догађајно оријентисани приступ где промене у различитим табелама (*History table*, *Subscriptions table*, *Likes table*) аутоматски покрећу одговарајуће *Lambda* функције:

- **Feed based on history:** анализира историју гледања корисника,
- **Feed based on subs:** узима у обзир претплате корисника на одређене теме или канале,
- **Feed based on likes:** укључује филмове сличне онима које је корисник означио као омиљене.

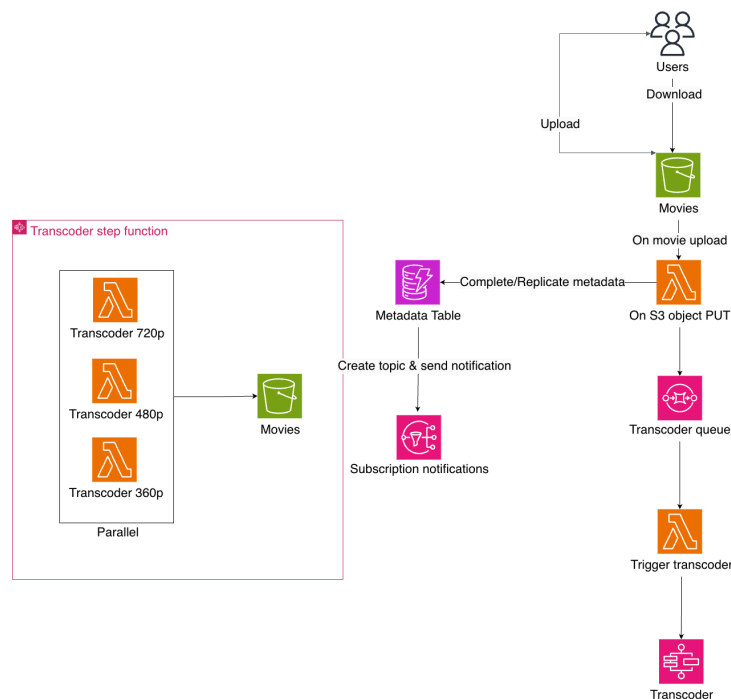
Све три *Lambda* функције шаљу препоруке у заједнички *Feed queue (SQS)*, одакле *Personalized feed Lambda* функција обједињава резултате и уписује финални јединствен и индивидуалан *feed* у *Feed table*. Овај асинхрони приступ омогућава да генерисање *feed*-а не блокира друге операције и да се може лако скалирати.

4.1.3 Архитектура транскодирања

Транскодирање видео садржаја представља ресурсно интензиван процес који захтева посебну обраду (Слика 6).

Процес транскодирања започиње када корисник постави нови филм у *Movies S3 bucket*. Догађај постављања (*On movie upload*) аутоматски покреће *On S3 object PUT Lambda* функцију која:

1. Припрема метаподатке о филму и уписује их у *Metadata Table*
2. Шаље поруку у *Transcoder queue (SQS)* са информацијама о филму који треба транскодирати
3. Креира тему у *SNS* сервису и шаље обавештење о почетку обраде преко *Stream processor Lambda* функције



Слика 6: Архитектура транскодирања филмова.

Trigger transcoder Lambda функција преузима поруке из *Transcoder queue*-а и покреће *Transcoder AWS Step Function*, који паралелно генерише ниже резолуције филмова. Паралелно извршавање транскодирања значајно убрзава процес обраде.

Резултати транскодирања складиште се назад у *Movies S3 bucket* у одговарајућим директоријумима. *Transcoder step function* координира целокупан процес и обезбеђује да се све фазе изврше редоследно и без грешака.

4.2 CI/CD pipeline архитектура

CI/CD pipeline за streaming апликацију имплементиран је помоћу AWS CodePipeline и AWS CodeBuild сервиса, уз коришћење AWS CDK. CDK омогућава декларативно описивање pipeline-а, након чега генерише одговарајуће AWS CloudFormation шаблоне који креирају и конфигуришу CodePipeline и CodeBuild ресурсе, као и све ресурсе који чине саму streaming платформу.

Pipeline је организован као низ фаза које се извршавају секвенцијално, при чему свака фаза мора успешно да се заврши пре него што процес настави даље. Архитектура pipeline-а је приказана на слици 7.

4.2.1 Извор и покретање pipeline-а

Процес започиње променом кода у GitHub репозиторијуму на develop грани. AWS CodePipeline аутоматски детектује промене преко webhook механизма и иницира извршавање pipeline-а. Ова фаза (Source Stage) преузима најновију верзију кода из репозиторијума и прослеђује је наредним фазама као улазни артефакт.

4.2.2 Изградња и синтеза инфраструктуре

Фаза изградње (Build) и ажурирања pipeline-а (Update Pipeline), приказане на дијаграму, представљају битан део CodePipeline-а. Током ових фаза, AWS CodeBuild извршава команде за инсталацију зависности и покретање CDK синтезе. Резултат синтезе је генерисање CloudFormation шаблона који описује комплетну инфраструктуру апликације.

Након синтезе, CodePipeline аутоматски извршава фазу ажурирања pipeline-а (self-mutation), при чему се све промене у дефиницији pipeline-а, описане у CDK коду, примењују на сам pipeline пре наставка даљег извршавања. Овим механизмом обезбеђује се да структура CI/CD pipeline-а увек буде синхронизована са његовом декларативном дефиницијом.

Поред тога, у оквиру ове фазе врши се и објављивање (Publish Assets) свих апликационих артефаката, као што су пакети Lambda функција и други статички ресурси, чиме се обезбеђује њихова доступност током каснијих фаза постављања.

4.2.3 Фаза контроле квалитета

Пре постављања у било које окружење, код пролази кроз две паралелне провере квалитета:

SonarQube анализа: Алат SonarQube врши статичку анализу кода и детектује потенцијалне проблеме као што су грешке, безбедносне рањивости, дубликација кода и неусаглашеност са кодинг стандардима. Анализа се извршава у одвојеном AWS CodeBuild окружењу.

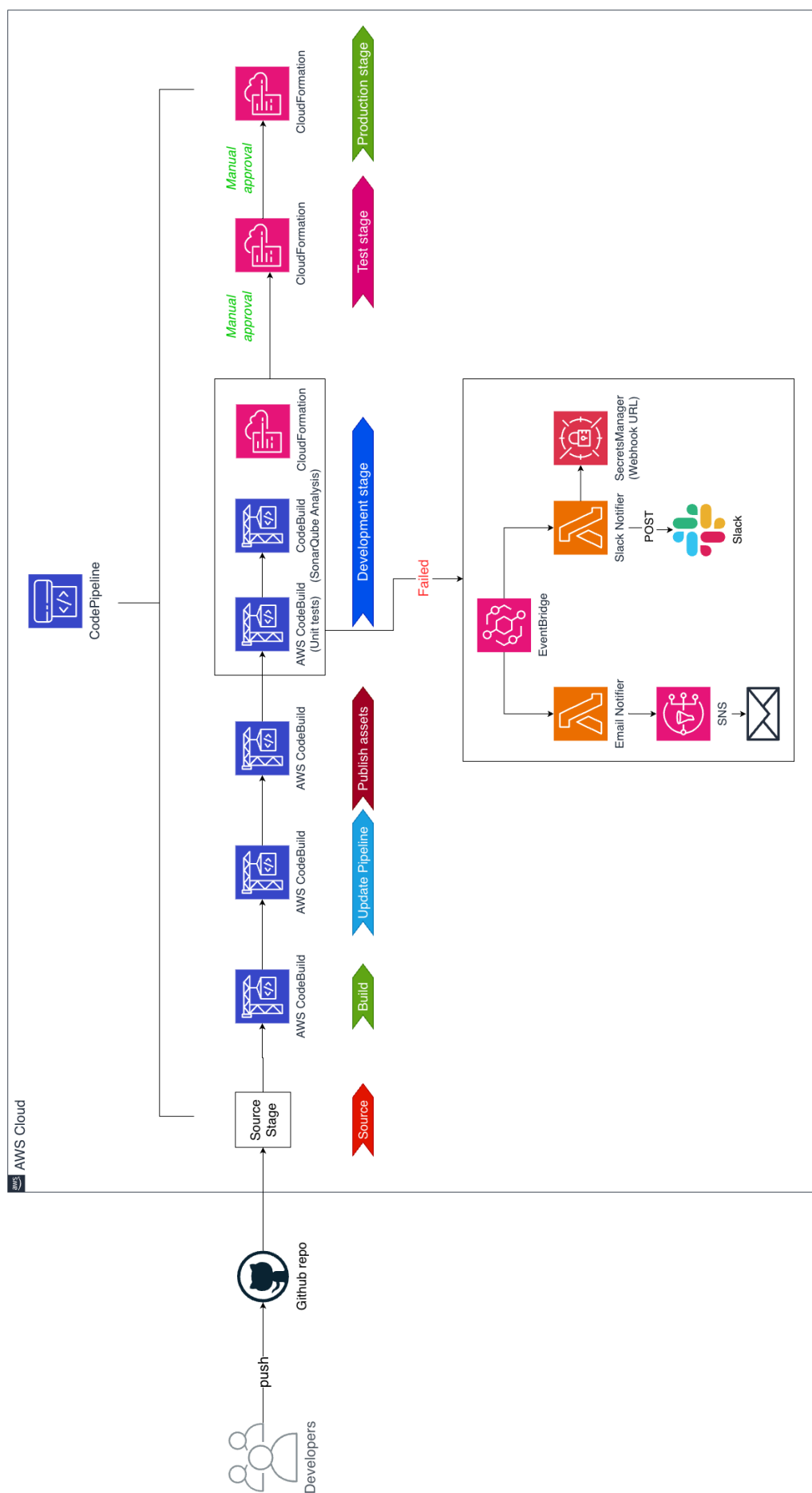
Јединични тестови: Паралелно са SonarQube анализом, извршавају се јединични тестови (енгл. unit tests) који валидирају функционалност појединачних Lambda функција у изолованом окружењу. Тестови користе pytest радни оквир и симулирају AWS окружење постављањем променљивих окружења које имитирају развојно окружење.

Обе провере морају успешно да се заврше како би pipeline наставио даље. Ако било која од њих не успе, извршавање pipeline-а се зауставља и програмери добијају обавештење о проблему.

4.2.4 Постављање у развојно окружење

Након успешних провера квалитета, pipeline улази у фазу постављања у развојно окружење (енгл. Development stage). AWS CodeBuild извршава CloudFormation команде за креирање или ажурирање ресурса у развојном AWS налогу. Ова фаза аутоматски поставља све компоненте система (Lambda функције, API Gateway, DynamoDB табеле, S3 bucket-е, SNS теме, SQS редове итд.) - према дефиницији из CDK кода.

Након успешног постављања, CloudFormation извршава провере да би верификовао да су сви ресурси правилно креирани.



Слика 7: Архитектура CI/CD pipeline-а са фазама изградње, тестирања и постављања.

4.2.5 Ручно одобрење за тестно окружење

Прелазак на тестно окружење (QA) захтева ручно одобравање. Корак ручног одобравања (енгл. *Manual Approval*) паузира извршавање *pipeline*-а и чека да овлашћена особа прегледа промене и одобри постављање. Током ове паузе, програмери и тестери могу детаљно тестирати апликацију у развојном окружењу, валидирати нове функционалности и проверити да ли постоје проблеми.

Одобрење се даје преко AWS конзоле или *CLI*-а, након чега *pipeline* наставља са постављањем у QA окружење које је идентично по конфигурацији развојном окружењу, али користи засебне AWS ресурсе и базе података.

4.2.6 Тестно окружење

Тестно окружење служи као последњи корак валидације пре продукције. У овом окружењу извршавају се интеграциони тестови, *end-to-end* тестови, тестови оптерећења и ручно тестирање од стране QA тима. Ово окружење симулира продукционе услове и омогућава откривање проблема који се не могу детектовати јединичним тестовима.

4.2.7 Ручно одобрење за продукцију

Пре коначног постављања у продукцију, *pipeline* поново захтева ручно одобрење. Ова одлука је критична јер директно утиче на крајње кориснике. Одобрење обично даје особа са одговорношћу за продукционо окружење, након што је валидирано да QA тестирање није открило критичне проблеме.

4.2.8 Продукционо окружење

Финална фаза *pipeline*-а поставља апликацију у продукционо окружење где јој приступају крајњи корисници. Продукционо окружење користи строжије безбедносне политике, већу алокацију ресурса за боље перформансе и напредније механизме надзора и алармирања. Све промене у продукцији евидентирају се (енгл. *logging*) и прате, омогућавајући брз опоравак (енгл. *rollback*) у случају проблема.

4.2.9 Систем обавештавања

Поред главног тока *pipeline*-а, архитектура укључује и посебан систем обавештавања који информисе тим о стању *pipeline*-а. Овај систем се састоји од неколико компоненти:

EventBridge правила: AWS *EventBridge* прати догађаје који се дешавају у *CodePipeline* и *CodeBuild* сервисима. Правила су конфигурисана да детектују промене стања *pipeline*-а (покретање, успех, неуспех) у *CodeBuild* фазама.

Lambda функције за обавештавање: Када *EventBridge* детектује релевантан догађај, активира одговарајуће *Lambda* функције које форматирају поруке и шаљу их кроз различите канале:

- *Slack Notifier Lambda*: преузима *webhook URL* из *AWS Secrets Manager*-а и шаље обавештења у *Slack* канал тима о покретању, успеху или неуспеху *pipeline*-а,
- *Email Notifier Lambda*: у случају неуспеха *pipeline*-а или *CodeBuild* фазе, објављује поруку на *SNS* тему која је повезана са имејл претплатама.

Овај систем обезбеђује да тим увек буде информисан о стању *pipeline*-а, омогућавајући брзу реакцију на проблеме и смањујући време између детекције грешке и њене исправке.

У овом поглављу детаљно је описана имплементација *CI/CD pipeline*-а за *streaming* апликацију, укључујући конфигурацију *AWS* сервиса, дефинисање инфраструктуре као кода, имплементацију система обавештавања и интеграцију алата за контролу квалитета кода.

5.1 Структура пројекта и *CDK* апликација

Пројекат је организован као *AWS CDK* апликација, при чему је инфраструктурни код имплементиран у програмском језику *TypeScript*. Оваква организација омогућава поновљивост, лакше конфигурисање и лакше одржавање *CI/CD pipeline*-а.

5.1.1 Организација пројекта

Пројекат је организован у следећу структуру приказану на листингу 1:

```
infrastructure/
├── bin /
│   └── infrastructure.ts.    // Улазна тачка пројекта
├── lib/
│   ├── cicd-stack.ts        // CI/CD pipeline дефиниција
│   ├── pipeline-stage.ts    // Stage за постављање
│   ├── stacks/              // Апликациони stack-ови
│   └── constructs/          // Апликациони конструкти
├── lambda/
│   └── event-invoked/
│       ├── cicd-notifier.py  // Email notifier Lambda
│       └── cicd-slack-notifier.py // Slack notifier Lambda
├── test/
│   └── unit/                 // Јединични тестови
├── cdk.json                  // CDK конфигурација
└── package.json              // Node.js зависности
```

Листинг 1: Структура пројекта.

5.1.2 Иницијализација *CDK* апликације

Листинг 2 приказује улазну тачку *CDK* апликације која креира *CicdStack*:

```
#!/usr/bin/env node
import 'source-map-support/register';
import * as cdk from 'aws-cdk-lib';
import { CicdStack } from '../lib/cicd-stack';

const app = new cdk.App();

new CicdStack(app, 'CicdStack', {
  env: {
    account: process.env.CDK_DEFAULT_ACCOUNT,
    region: process.env.CDK_DEFAULT_REGION
  },
});

app.synth();
```

Листинг 2: Иницијализација *CDK* апликације.

5.2 Имплементација *CI/CD Pipeline Stack-a*

Централна компонента система је *CicdStack* класа која дефинише комплетан *CI/CD pipeline* користећи *AWS CDK* конструкте за креирање *CodePipeline* и *CodeBuild* ресурса.

Листинг 3 приказује заглавље *CicdStack* класе, која наслеђује *cdk.Stack* класу и представља логичку целину за дефинисање *pipeline* инфраструктуре. Конструктор класе прима основне параметре неопходне за иницијализацију *stack-a*, након чега се у наставку дефинишу појединачне фазе и кораци *pipeline-a*.

```
export class CicdStack extends cdk.Stack {
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {
    super(scope, id, props);
    // ...
  }
}
```

Листинг 3: Заглавље *CicdStack* класе.

5.2.1 Креирање *CodePipeline-a*

Листинг 4 приказује креирање *pipeline-a* са дефиницијом извора кода и фазе синтезе:

```
const pipeline = new CodePipeline(this, 'Pipeline', {
  pipelineName: 'Pipeline',
  synth: new ShellStep('Synth', {
    input: CodePipelineSource.github(
      'duledjordjevic/streamio',
      'develop'
    ),
    commands: [
      'cd infrastructure',
      'npm ci',
      'npx cdk synth',
    ],
    primaryOutputDirectory: 'infrastructure/cdk.out'
  })
});
```

Листинг 4: Креирање *CodePipeline-a* са *GitHub* извором

CodePipeline конструкт аутоматски генерише *AWS CodePipeline* ресурс који укључује фазе за преузимање кода, синтезу *CloudFormation* шаблона и самоажурирање *pipeline-a*.

Извор кода конфигурисан је као *GitHub* репозиторијум са гране *develop*, чиме се омогућава аутоматско покретање *pipeline-a* при свакој промени кода у дефинисаној развојној грани.

Фаза синтезе извршава *cdk synth* команду, којом се *TypeScript* дефиниције инфраструктуре трансформишу у *CloudFormation* шаблоне. На овај начин обезбеђује се да је инфраструктура као код увек репродуктивна и верзионисана заједно са апликационим изворним кодом.

5.2.2 Интеграција *SonarQube* анализе

SonarQube анализа имплементирана је као *CodeBuildStep* који се извршава пре постављања у развојно окружење. Конфигурација анализе приказана је на листингу 5.

Током извршавања *CodeBuild* корака, у радном окружењу се динамички преузима и распакује алат *sonar-scanner*. Након распакивања, путања до извршне датотеке додаје се у системску променљиву *PATH*, што омогућава једноставно покретање алата у наставку процеса.

Сам процес анализе иницира се командом *sonar-scanner*, којом се врши локална анализа изворног кода у оквиру *CodeBuild* окружења. Анализа обухвата проверу квалитета кода, откривање потенцијалних грешака, безбедносних рањивости и одступања од дефинисаних правила. Резултати анализе се након тога шаљу ка *SonarQube* серверу, где се централизовано обрађују и приказују кориснику.

Аутентификација према *SonarQube* платформи реализована је коришћењем приступног токена (*SONAR_TOKEN*). Овај токен се безбедно чува у сервису *AWS Secrets Manager* и током извршавања *CodeBuild* пројекта динамички се умета као променљива окружења. На овај начин се избегава чување осетљивих података у изворном коду или конфигурационим датотекама, чиме се постиже усклађеност са најбољим безбедносним праксама.

У случају да дефинисани услови *quality gate*-а нису испуњени, *pipeline* се прекида, чиме се спречава да код који не задовољава задате стандарде буде даље постављен у циљно окружење.

```
const sonarToken = secretsmanager.Secret.fromSecretNameV2(this, 'SonarToken', 'sonarqube-token');
const sonarQubeStep = new CodeBuildStep('SonarQube Analysis', {
  commands: [
    'curl --create-dirs -sSLo $HOME/.sonar/sonar-scanner.zip $SONAR_SCANNER_URL',
    'unzip -o $HOME/.sonar/sonar-scanner.zip -d $HOME/.sonar/',
    'export PATH=$SONAR_SCANNER_HOME/bin:$PATH',

    'sonar-scanner ' +
    '-Dsonar.projectKey=$SONAR_PROJECT_KEY ' +
    '-Dsonar.organization=$SONAR_ORGANIZATION ' +
    '-Dsonar.sources=. ' +
    '-Dsonar.host.url=$SONAR_HOST_URL ' +
    '-Dsonar.token=$SONAR_TOKEN ' +
    '-Dsonar.qualitygate.wait=true ' +
    '-Dsonar.ws.timeout=300'
  ],
  buildEnvironment: {
    environmentVariables: {
      SONAR_TOKEN: {
        type: cdk.aws_codebuild.BuildEnvironmentVariableType.SECRETS_MANAGER,
        value: sonarToken.secretArn
      },
      SONAR_SCANNER_URL: {
        value: 'https://binaries.sonarsource.com/Distribution/sonar-scanner-cli-7.2.0.5079-linux-x64.zip'
      },
      SONAR_SCANNER_HOME: { value: '/root/.sonar/sonar-scanner-7.2.0.5079-linux-x64' },
      SONAR_PROJECT_KEY: { value: 'duledjordjevic_streamio' },
      SONAR_ORGANIZATION: { value: 'duledjordjevic' },
      SONAR_HOST_URL: { value: 'https://sonarcloud.io' }
    }
  }
});
```

Листинг 5: Конфигурација *SonarQube* анализе.

5.2.3 Интеграција јединичних тестова

Јединични тестови се извршавају паралелно са *SonarQube* анализом. Листинг 6 приказује *CodeBuildStep* у коме се извршавају јединични тестови:

```
devStage.addPre(new CodeBuildStep('unit tests', {
  commands: [
    'cd infrastructure/test/unit',
    'pip install -r requirements.txt',
    'pytest -q test_upload_url.py',
  ],
  buildEnvironment: {
    environmentVariables: {
      MOVIES_BUCKET: { value: 'streamio-movies-bucket' },
      METADATA_TABLE: { value: 'StreamioMetadata' },
      // остале променљиве...
    }
  }
}));
```

Листинг 6: Интеграција јединичних тестова у *pipeline*.

Тестови су написани у *Python*-у користећи *pytest* радни оквир и симулирају развојно *AWS* окружење постављањем одговарајућих променљивих окружења.

5.2.4 Дефинисање *deployment* фаза

Pipeline укључује три окружења (развојно, тестно и продукционо) са различитим нивоима контроле. Свако окружење представља засебан *AWS* налог или изоловану групу ресурса. Листинг 7 приказује дефиницију свих фаза:

```
// Развојно окружење - аутоматско постављање
const devStage = pipeline.addStage(
  new PipelineStage(this, 'PipelineDevStage', {
    stageName: 'dev'
  })
);

devStage.addPre(sonarQubeStep);
devStage.addPre(unitTestsStep);

// Тестно окружење - са ручним одобрењем
const qaStage = pipeline.addStage(
  new PipelineStage(this, 'PipelineQAStage', {
    stageName: 'qa'
  })
);
qaStage.addPre(new cdk.pipelines.ManualApprovalStep('ApproveQA'));

// Продукционо окружење - са ручним одобрењем
const prodStage = pipeline.addStage(
  new PipelineStage(this, 'PipelineProdStage', {
    stageName: 'prod'
  })
);
prodStage.addPre(new cdk.pipelines.ManualApprovalStep('ApproveProd'));
```

Листинг 7: Дефинисање *deployment* фаза.

Метода `addPre()` додаје кораке који се извршавају пре постављања у одређено окружење. Када се више корака дода помоћу `addPre()`, они се извршавају паралелно, чиме се убрзава процес. `ManualApprovalStep` креира паузу за ручну проверу и одобрење, при чему овлашћена особа прегледа промене преко *AWS* конзоле пре наставка *pipeline*-а.

Развојно окружење се аутоматски ажурира након успешних провера квалитета (*SonarQube* и јединични тестови), док тестно и продукционо окружење захтевају ручно одобрење како би се обезбедила додатна контрола над променама.

5.3 Имплементација система обавештавања

Систем обавештавања омогућава тиму да прати статус *pipeline*-а у реалном времену преко *Slack*-а и имејла. Овај систем је критичан за брзу реакцију на проблеме.

5.3.1 *EventBridge* правила за детекцију догађаја

AWS EventBridge прати промене у *AWS* сервисима и покреће акције на основу унапред дефинисаних правила. У овој имплементацији, правила су конфигурирана да детектују промене у *CodePipeline* и *CodeBuild* сервисима. Листинг 8 приказује дефиницију *EventBridge* правила:

```
new events.Rule(this, 'CodePipelineFailedRule', {
  description: 'Notify on failed pipeline executions',
  eventPattern: {
    source: ['aws.codepipeline'],
    detailType: ['CodePipeline Pipeline Execution State Change'],
    detail: {
      state: ['FAILED'],
      pipeline: ['Pipeline'],
    },
  },
  targets: [new targets.LambdaFunction(notifierFn)],
});

new events.Rule(this, 'CodeBuildFailedRule', {
  description: 'Notify on failed codebuild builds',
  eventPattern: {
    source: ['aws.codebuild'],
    detailType: ['CodeBuild Build State Change'],
    detail: {
      'build-status': ['FAILED']
    },
  },
  targets: [new targets.LambdaFunction(notifierFn)],
});
```

Листинг 8: *EventBridge* правила за детекцију неуспеха.

Параметар `eventPattern` дефинише филтер који одређује који догађаји ће активирати правило. Одвојена правила за *CodePipeline* и *CodeBuild* обезбеђују да тим буде обавештен о неуспесима на свим нивоима *CI/CD* процеса, јер неуспеси у *CodeBuild* фазама не морају увек означити неуспех целог *pipeline*-а.

5.3.2 *Slack* интеграција

Slack обавештења имплементирана су помоћу *webhook* механизма. *Webhook URL* се безбедно чува у *AWS Secrets Manager*-у уместо у изворном коду, чиме се спречава изложеност осетљивих података. *Lambda* функција добија дозволу за читање помоћу `grantRead()` методе, која аутоматски креира одговарајућу *IAM* политику. Када *EventBridge* детектује релевантан догађај, активира *Lambda* функцију која форматира и шаље поруку у *Slack* канал. Листинг 9 приказује *Slack* интеграцију са *pipeline*-ом:

```
const slackWebhookUrl = secretsmanager.Secret.fromSecretNameV2(  
    this,  
    'slackWebhookUrl',  
    'slack-webhook-url'  
);  
  
const slackNotifier = new lambda.Function(this, 'SlackNotifier', {  
    runtime: lambda.Runtime.PYTHON_3_11,  
    handler: 'cicd-slack-notifier.handler',  
    timeout: cdk.Duration.seconds(20),  
    code: lambda.Code.fromAsset(  
        path.join(__dirname, '../lambda/event-invoked')  
    ),  
    environment: {  
        SLACK_WEBHOOK_URL: slackWebhookUrl.secretValue.unsafeUnwrap()  
    }  
});  
  
slackWebhookUrl.grantRead(slackNotifier);
```

Листинг 9: Slack интеграција помоћу *webhook* механизма.

5.3.3 Имејл обавештења

Имејл обавештења користе SNS сервис за дистрибуцију порука. Листинг 10 приказује интеграцију имејл обавештења:

```
const emailNotifyTopic = new sns.Topic(this,  
    'PipelineNotificationsTopic', {  
        displayName: 'Pipeline notifications (streamio)',  
        topicName: 'streamio-pipeline-notifications',  
    });  
  
emailNotifyTopic.addSubscription(  
    new subs.EmailSubscription('djordjevicdusan24@gmail.com')  
);  
  
const notifierFn = new lambda.Function(this, "NotifierFn", {  
    runtime: lambda.Runtime.PYTHON_3_11,  
    handler: 'cicd-notifier.handler',  
    code: lambda.Code.fromAsset(  
        path.join(__dirname, '../lambda/event-invoked')  
    ),  
    environment: {  
        TARGET_TOPIC: emailNotifyTopic.topicArn  
    }  
});  
  
emailNotifyTopic.grantPublish(notifierFn);
```

Листинг 10: Интеграција имејл обавештења преко SNS теме.

SNS тема служи као централна тачка за дистрибуцију порука, омогућавајући да више претплатника прима исте поруке. *Lambda* функција добија дозволу за објављивање порука на SNS тему помоћу `grantPublish()` методе. Систем користи две одвојене *Lambda* функције: *Slack* обавештења шаљу се за све промене стања *pipeline*-а (покретање, успех, неуспех), док се имејл обавештења шаљу само за неуспехе.

У овом раду приказана је практична примена *CI/CD* механизма за аутоматизацију испоруке *serverless* сервиса на *AWS* платформи. Имплементирано решење обухвата комплетан *pipeline* за изградњу, тестирање и постављање *streaming* апликације у више окружења, уз примену инфраструктуре као кода и догађајно оријентисаног система обавештавања.

6.1 Предности имплементираног решења

Предност овог приступа јесте доследност окружења. Применом инфраструктуре као кода, сва окружења се постављају на идентичан начин, што елиминише проблеме који настају услед различитих конфигурација. Аутоматизација процеса изградње и постављања значајно смањује време потребно за испоруку нових функционалности и исправки грешака. Догађајно оријентисан систем обавештавања омогућава брзу реакцију на проблеме, док интеграција *SonarQube* анализе и јединичних тестова обезбеђује рану детекцију грешака.

6.2 Препознати недостаци

Иако имплементирано решење доноси бројне предности, постоје и одређени недостаци. *Serverless* архитектура отежава дијагностиковање узрока проблема, посебно када грешка настаје услед интеракције између више сервиса. Време извршавања *pipeline*-а могло би бити оптимизовано, док зависност од *AWS* платформе представља дугорочни ризик. Тренутна имплементација такође не укључује детаљно праћење перформанси система након постављања.

6.3 Правци даљег развоја

Систем би могао бити унапређен имплементацијом напредног надгледања и аларма, увођењем аутоматизованих тестова оптерећења и применом *blue-green* или *canary deployment* стратегија. Ова унапређења би значајно повећала поузданост и смањила ризик приликом испоруке нових верзија апликације.

Континуирана интеграција и испорука представљају кључне праксе за одрживост савремених софтверских система. Имплементирано решење показује да је могуће изградити поуздан, аутоматизован и скалабилан процес испоруке користећи *AWS* алате, обезбеђујући добру основу за даљи развој *streaming* платформе.

Списак слика

Слика 1	Поређење <i>IaaS</i> , <i>PaaS</i> и <i>SaaS</i> модела према нивоу управљања ресурсима. [12] .	4
Слика 2	Пример <i>AWS serverless</i> архитектуре [14] .	5
Слика 3	<i>CI/CD</i> циклус [16] .	7
Слика 4	<i>REST API</i> архитектура.	14
Слика 5	Архитектура <i>Feed</i> система.	15
Слика 6	Архитектура транскодирања филмова.	16
Слика 7	Архитектура <i>CI/CD pipeline</i> -а са фазама изградње, тестирања и постављања.	18

Списак листинга

Листинг 1	Структура пројекта.	21
Листинг 2	Иницијализација <i>CDK</i> апликације.	21
Листинг 3	Заглавље <i>CicdStack</i> класе.	22
Листинг 4	Креирање <i>CodePipeline</i> -а са <i>GitHub</i> извором.	22
Листинг 5	Конфигурација <i>SonarQube</i> анализе.	23
Листинг 6	Интеграција јединичних тестова у <i>pipeline</i>	23
Листинг 7	Дефинисање <i>deployment</i> фаза.	24
Листинг 8	<i>EventBridge</i> правила за детекцију неуспеха.	25
Листинг 9	<i>Slack</i> интеграција помоћу <i>webhook</i> механизма.	26
Листинг 10	Интеграција имејл обавештења преко <i>SNS</i> теме.	26

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CDK	Cloud Development Kit (комплет алата за развој у облаку)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CLI	Command Line Interface (интерфејс командне линије)
CRUD	Create, Read, Update, Delete (креирање, читање, ажурирање, брисање)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
IaaS	Infrastructure as a Service (инфраструктура као сервис)
JSON	JavaScript Object Notation(формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
PaaS	Platform as a Service (платформа као сервис)
QA	Quality Assurance (обезбеђење квалитета)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
S3	Simple Storage Service (сервис за складиштење података)
SaaS	Software as a Service (софтвер као сервис)
SNS	Simple Notification Service (сервис за обавештавање)
SQS	Simple Queue Service (сервис за обавештавање уз помоћ редова)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)

Списак коришћених појмова

Појам	Објашњење
Артефакт	Излазни производ процеса изградње софтвера, као што су извршне датотеке, библиотеке или пакети.
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Blue-green deployment	Стратегија постављања где се нова верзија апликације поставља паралелно са постојећом, са тренутном променом саобраћаја.
Canary release	Стратегија постепеног постављања где се нова верзија апликације прво испоручује малом проценту корисника пре потпуног постављања.
CloudFormation	AWS сервис за дефинисање и управљање инфраструктуром помоћу шаблона.
Code smell	Индикатор потенцијалног проблема у коду који указује на лош дизајн или погрешну имплементацију.
Догађајно оријентисана архитектура	Архитектурни приступ где компоненте система реагују на догађаје, омогућавајући лабаво повезивање.
Lambda функција	Безсервска функција која се извршава као одговор на догађаје без потребе за управљањем серверима.
Микросервис	Архитектурни приступ где апликација се гради као скуп малих, независних сервиса.
Pipeline	Низ аутоматизованих корака за изградњу, тестирање и постављање софтвера.
Presigned URL	Привремени URL који омогућава ограничен приступ ресурсима без потребе за аутентификацијом.
Rollback	Процес враћања система на претходну стабилну верзију након откривања проблема.
Serverless	Модел извршавања где cloud провајдер управља инфраструктуром, а корисник плаћа само за стварно коришћене ресурсе.
Stack	Логичка целина AWS ресурса којом се управља као једна јединица.
Транскодирање	Процес конверзије мултимедијалног садржаја из једног формата или резолуције у други.
Webhook	Механизам за аутоматско слање HTTP обавештења када се деси одређени догађај.

Биографија

Душан Ђорђевић је рођен 16.04.2002. године у Врању, Србија. Основну школу „Бранко Радичевић“ завршио је 2017. године. Гимназију „Бора Станковић“ завршио је 2021. године. Исте године уписао се на Факултет техничких наука, смер Софтверско инжењерство и информационе технологије. Положио је све испите предвиђене планом и програмом.

Литература

- [1] F. P. B. Jr., „No Silver Bullet – Essence and Accident in Software Engineering“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://web.archive.org/web/20160910002130/http://worrydream.com/refs/Brooks-NoSilverBullet.pdf>
- [2] Atlassian, „Continuous Integration vs. Delivery vs. Deployment“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>
- [3] B. Beyer, C. Jones, J. Petoff, и N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. [На Интернету]. Доступно на <https://sre.google/books/>
- [4] Amazon Web Services, „Serverless Architectures - Learn More“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://aws.amazon.com/lambda/serverless-architectures-learn-more/>
- [5] K. Morris, *Infrastructure as Code*. O'Reilly Media, 2021. [На Интернету]. Доступно на <https://books.google.rs/books?id=UW4NEAAQBAJ>
- [6] S. Chinamanagonda, „Automating Infrastructure with Infrastructure as Code (IaC)“, *SSRN Electronic Journal*, 2019, doi: [10.21275/SR24829170834](https://doi.org/10.21275/SR24829170834).
- [7] A. S. Tanenbaum и M. V. Steen, *Distributed Systems: Principles and Paradigms*, 3rd изд. CreateSpace Independent Publishing Platform, 2017.
- [8] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017. [На Интернету]. Доступно на <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>
- [9] J. Lewis и M. Fowler, „Microservices: a definition of this new architectural term“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://martinfowler.com/articles/microservices.html>
- [10] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018. [На Интернету]. Доступно на <https://www.manning.com/books/microservices-patterns>
- [11] C. D. Nguyen, „A Design Analysis of Cloud-based Microservices Architecture at Netflix“, *The Startup, Medium*, 2020, Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://medium.com/swlh/a-design-analysis-of-cloud-based-microservices-architecture-at-netflix-98836b2da45f>
- [12] Red Hat, „IaaS vs. PaaS vs. SaaS“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://www.redhat.com/en/topics/cloud-computing/iaas-vs-paas-vs-saas>
- [13] R. Rajan, „Serverless Architecture: A Survey“, *IEEE International Conference on Cloud Computing Technology and Science*, стр. 1–8, 2018.
- [14] Andrew Robinson, „Liberty IT Adopts Serverless Best Practices Using AWS Cloud Development Kit“. Приступљено: 22. Децембар 2025. [На Интернету]. Доступно на <https://aws.amazon.com/blogs/architecture/liberty-it-adopts-serverless-best-practices-using-aws-cdk/>
- [15] M. M. P. P. Garima Bajpai Michel Schildmeijer, *CI/CD Design Patterns*. O'Reilly Media, 2024. [На Интернету]. Доступно на https://books.google.com/books/about/CI_CD_Design_Patterns.html?id=aykyEQAAQBAJ
- [16] BlackDuck, „Continuous Integration and Continuous Delivery/Deployment“. Приступљено: 22. Децембар 2025. [На Интернету]. Доступно на <https://www.blackduck.com/glossary/what-is-cicd.html>

-
- [17] ISO, „ISO/IEC/IEEE 32675:2022 Information technology — DevOps — Building reliable and secure systems including application build, package and deployment“. Приступљено: 22. Децембар 2025. [На Интернету]. Доступно на <https://www.iso.org/standard/83670.html>
- [18] HashiCorp, „What is Infrastructure as Code with Terraform?“. Приступљено: 17. Децембар 2025. [На Интернету]. Доступно на <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>
- [19] М. Kolny, „Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%“. Приступљено: 17. Јануар 2025. [На Интернету]. Доступно на <https://www.wudsn.com/productions/www/site/news/2023/2023-05-08-microservices-01.pdf>